

CP020003

ARTIFICIAL INTELLIGENCE



Khon Kaen University · 2026

Week 11

Computer Vision

Teaching a Model to Read Pictures

Six tasks, one shared recipe: classify, detect (box & rotated box), segment, estimate depth, and find pose — all with the same train → predict → evaluate loop.



Instructor: Teerapong Panboonyuen (P'Kao)

https://github.com/kaopanboonyuen/CP020003_ArtificialIntelligence_2026s1



What We Will Cover



Image Classification

"What is this?" — one label per image



Object Detection (BBox)

"What, and where?" — upright boxes



Oriented Detection (OBB)

"Where, and at what angle?" — rotated boxes



Segmentation

"Which pixels belong to what?" — per-pixel masks



Depth Estimation

"How far away?" — a distance per pixel



Pose Estimation

"Where are the joints?" — a skeleton per person



What Is Computer Vision?

Computer Vision is the field of teaching software to make sense of visual data — photos, video frames, satellite images, X-rays — the way our eyes and brain do it for us. **Last week our model learned to read numbers (digits). This week it learns to read pictures.**

Every task starts the same way, then splits:

1

Pixels in

A grid of numbers (Red-Green-Blue values) — that is literally all a computer ever sees.

2

Backbone extracts features

A CNN (convolutional neural network) scans the grid and learns edges → shapes → parts → objects.

3

Task-specific "head"

The same features feed a different output layer depending on the question you ask — a label, a box, a mask, a depth map, or a skeleton.



A Human Analogy

Classification is glancing at a photo and saying "that's a dog."

Detection is pointing at each dog and each cat in a busy photo.

Segmentation is tracing the exact outline of each dog with your finger.

Depth is judging which dog is closer to you.

Pose is noticing exactly how the dog is standing — legs, head, tail.

COMPUTER VISION TASKS

CLASSIFICATION



CAT

CLASSIFICATION
+ LOCALIZATION



CAT

OBJECT
DETECTION



CAT, DOG, DUCK

INSTANCE
SEGMENTATION



CAT, DOG, DUCK

SINGLE OBJECT

MULTIPLE OBJECTS



truck #3

person #7

truck #4

forklift #6

person #8

The ultimate end-to-end computer vision platform

Annotate data, train YOLO models, and deploy to 42 global regions, from one unified platform.

Contact sales →

Get started

truck #5

person #9

person #10

Trusted by the world's leading organizations



PHILIPS

NEURA
ROBOTICS



TATA STEEL

Flock Safety

intel.



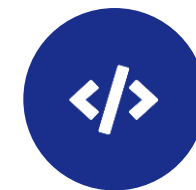
Ask AI



One Family, Six Questions

Task	Question It Answers	Output
 Classification	What is this?	One label for the whole image
 Detection (BBox)	What objects, and where?	Axis-aligned boxes + labels
 OBB	Where, and at what angle?	Rotated boxes + labels
 Segmentation	Which pixels belong to what?	Per-pixel masks
 Depth Estimation	How far away is each pixel?	A depth value per pixel
 Pose Estimation	Where are the joints?	A skeleton of keypoints

We will build a working, trainable, measurable pipeline for all six — using Ultralytics (the YOLO library) for five of them, plus a pretrained depth model for the one YOLO does not cover.



One Pattern That Repeats Everywhere

Once this loop clicks, switching tasks — or datasets — is mostly about getting your labels into the right format.



TASK 1

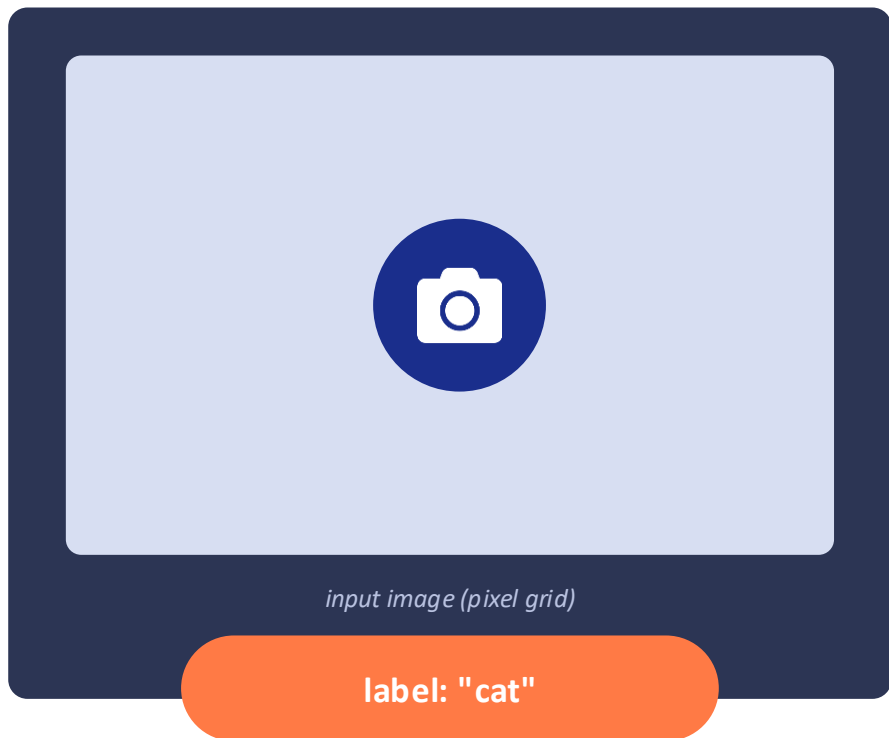
Image Classification

"What is this?" — the hello-world of computer vision.





Input & Label: One Tag Per Image



One label describes the entire picture — nothing about position or size.

Dataset layout — "ImageFolder" style

```
my_dataset/  
  train/  
    cat/      img001.jpg, img002.jpg ...  
    dog/      img101.jpg, img102.jpg ...  
    bird/     ...  
  val/  
    cat/  dog/  bird/
```

No YAML needed — class names come straight from the sub-folder names.

Toy dataset used in class: mnist160 (160 images, handwritten digits 0–9)

```
model = YOLO("yolov8n-cls.pt")  
model.train(data="my_dataset", epochs=20, imgsz=224)
```



Where This Task Gets Used



Medical screening

Classify a scan as benign vs. malignant (e.g. skin lesion or tumor screening).



Content moderation

Flag NSFW, violent, or spam images at upload time.



Retail & e-commerce

Auto-tag product photos by category for search and catalogs.



Face & identity apps

Face recognition, emotion recognition, age/gender estimation.



Quality control

Sort factory-line products into "pass" / "defective" bins.



Agriculture

Classify a leaf photo as healthy or a specific crop disease.

Rule of thumb: reach for classification when you only need to know "what category is this whole image" — you don't care about position, count, or shape.

More examples: X-ray pneumonia screening · handwriting/digit recognition · plankton & insect species ID · satellite land-cover tiles · product-image auto-tagging.



How We Score It: Top-1 & Top-5 Accuracy

Standard since the ImageNet benchmark — both ask "was the correct label among the model's guesses?"

Top-1 Accuracy

Is the model's single best guess correct?

✓ Predicted #1 = "cat" — True label = "cat"

Formula:

$Top-1 = (\# \text{ images where the top guess is correct}) / (\text{total images})$

Strict — used when you need one confident, correct answer (e.g. a diagnosis).

Top-5 Accuracy

Is the true label among the model's 5 best guesses?

tiger

cat

lynx

fox

dog

Formula:

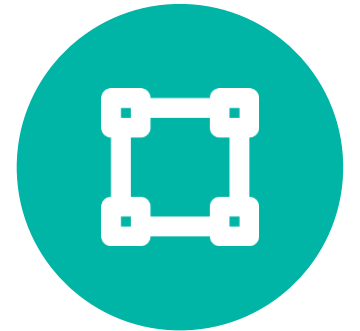
$Top-5 = (\# \text{ images where truth} \in \text{top-5 guesses}) / (\text{total images})$

More forgiving — useful when classes are visually similar (many dog or plant breeds).

TASK 2

Object Detection

"What objects, and where?" — upright boxes with labels.



The engine behind the platform. The standard in vision AI.

From YOLOv5 to YOLO26, Ultralytix builds the most widely adopted vision AI models, trusted by millions of developers and enterprises.

Object detection

Detect objects in an image or video with axis-aligned bounding boxes.

Instance segmentation

Semantic segmentation

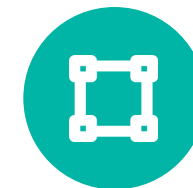
Depth estimation

Image classification

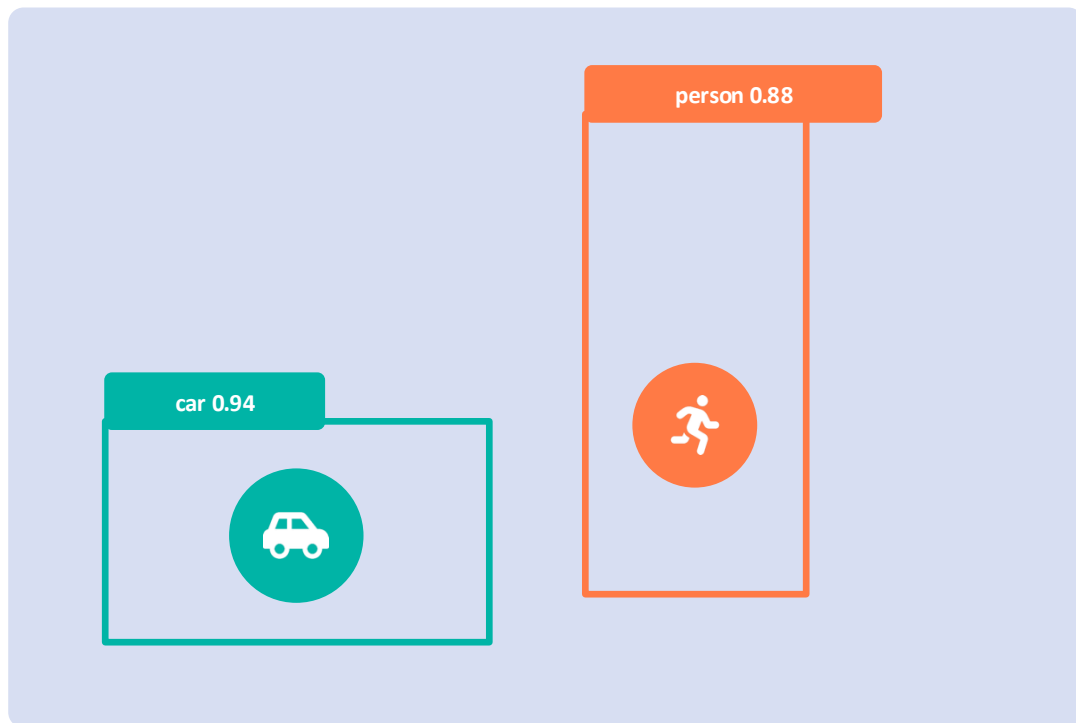
Pose estimation

Oriented object detection





Input & Label: Boxes Around Objects



Each box: a class label + confidence score + position/size.

Label file — one row per object (normalized 0–1)

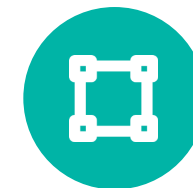
```
<class_id> <x_center> <y_center> <width> <height>  
0 0.532 0.481 0.220 0.340
```

class 0, centered at 53.2% / 48.1% of the image, spanning 22.0% of the width and 34.0% of the height.

Folder layout (same pattern for detection, OBB, segmentation, pose):

```
images/train, images/val  
labels/train, labels/val (same filename, .txt)  
+ my_dataset.yaml (paths + class names)
```

✂ Roboflow, CVAT, and LabelImg all export directly to this format.



Where This Task Gets Used



Traffic enforcement

Mount on a CCTV pole, detect cars, and flag ones exceeding the speed limit.



Retail & logistics

Count boxes on a conveyor, spot damaged packaging, self-checkout item recognition.



Autonomous driving

Detect pedestrians, vehicles, and signs in real time from a dash camera.



Medical imaging

Draw a box around a suspected tumor or fracture on an X-ray/CT slice.



Security & safety

Detect people/weapons in restricted areas; check workers are wearing helmets/PPE.



Wildlife monitoring

Count animals in camera-trap or drone footage for conservation research.

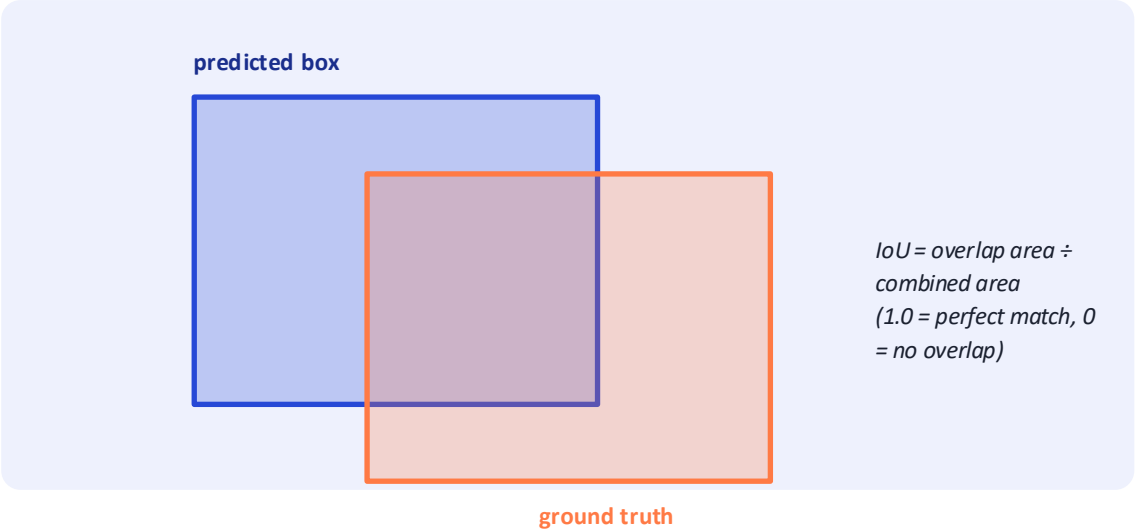
Rule of thumb: reach for detection when you need to know how many objects, what kind, and roughly where — but an upright rectangle is a good enough outline.

More examples: license-plate recognition · aerial surveying (buildings, crop rows) · crowd counting in stadiums · shelf-stock auditing in retail.



How We Score It: IoU & mAP

1. IoU — Intersection over Union



2. Precision & Recall (at a confidence threshold)

Precision = of the boxes we drew, how many were correct?

Recall = of the true objects, how many did we find?

A box only "counts" as a correct detection when its IoU with the true box clears a threshold (commonly 0.50).

3. mAP — mean Average Precision

Metric	IoU threshold	What it means
mAP50	IoU ≥ 0.50	A "loose" match — reasonably placed boxes count
mAP50-95	IoU 0.50 → 0.95, averaged	The strict, COCO-standard number — rewards tight boxes

TASK 3

Oriented Object Detection

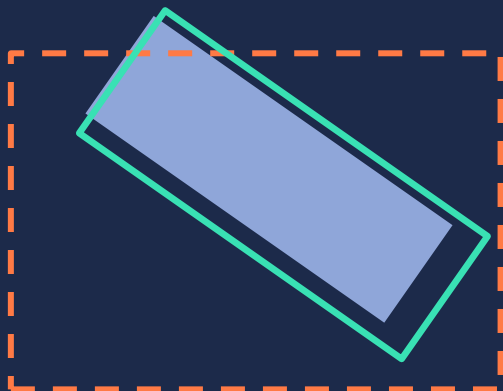
"Where, and at what angle?" — boxes that rotate to hug the object.





Why Rotate the Box?

Axis-aligned vs. Oriented (aerial photo of a ship)



--- axis-aligned box (wastes area)

— oriented box (hugs the ship)

Aerial/satellite imagery, rotated documents, or angled parking are where OBB earns its keep.

Label format — 4 corner points instead of center+size

```
<class_id> <x1> <y1> <x2> <y2> <x3> <y3> <x4> <y4>
```

Corners are listed in order (e.g. clockwise from top-left), normalized 0–1.

Toy dataset used in class: dota8 (8 aerial images from DOTA, ships / planes / vehicles)

```
model = YOLO("yolov8n-obb.pt")
```

Metric: same mAP50 / mAP50-95 family, but IoU is computed between rotated polygons — a harder geometry problem.

The engine behind the platform. The standard in vision AI.

From YOLOv5 to YOLO26, Ultralytix builds the most widely adopted vision AI models, trusted by millions of developers and enterprises.

Object detection

Instance segmentation

Semantic segmentation

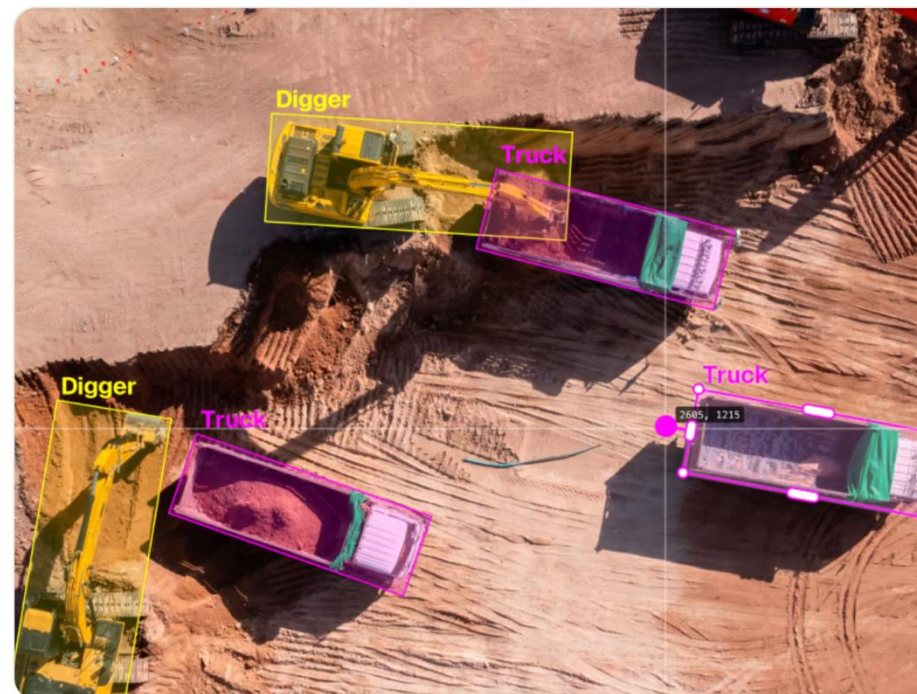
Depth estimation

Image classification

Pose estimation

Oriented object detection

Detect rotated objects with angle-aware bounding boxes.





Where This Task Gets Used



Satellite & aerial imagery

Count and classify ships, planes, or vehicles that sit at any angle.



Document scanning

Detect a tilted document or ID card so it can be auto-rotated & cropped.



Warehouse logistics

Locate pallets and packages placed at an angle on a shelf or floor.



Parking-lot analytics

Detect cars parked at an angle for occupancy counting.

Why not just use regular detection?

An axis-aligned box around a rotated object has to be big enough to contain the whole thing, so it captures a lot of background. That hurts anything that measures area, counts overlapping objects, or feeds a downstream size/shape estimate. OBB models exist as their own model family — not a post-processing trick — because computing IoU between rotated shapes is a genuinely harder geometry problem than between two rectangles.

TASK 4

Segmentation

"Which pixels belong to what?" — the outline hugs the exact silhouette.



The engine behind the platform. The standard in vision AI.

From YOLOv5 to YOLO26, Ultralytics builds the most widely adopted vision AI models, trusted by millions of developers and enterprises.

Object detection

Instance segmentation

Create pixel-precise masks highlighting objects within a frame.

Semantic segmentation

Depth estimation

Image classification

Pose estimation

Oriented object detection



The engine behind the platform. The standard in vision AI.

From YOLOv5 to YOLO26, Ultralytics builds the most widely adopted vision AI models, trusted by millions of developers and enterprises.

Object detection

Instance segmentation

Semantic segmentation

Assign a specific class label to every individual pixel.

Depth estimation

Image classification

Pose estimation

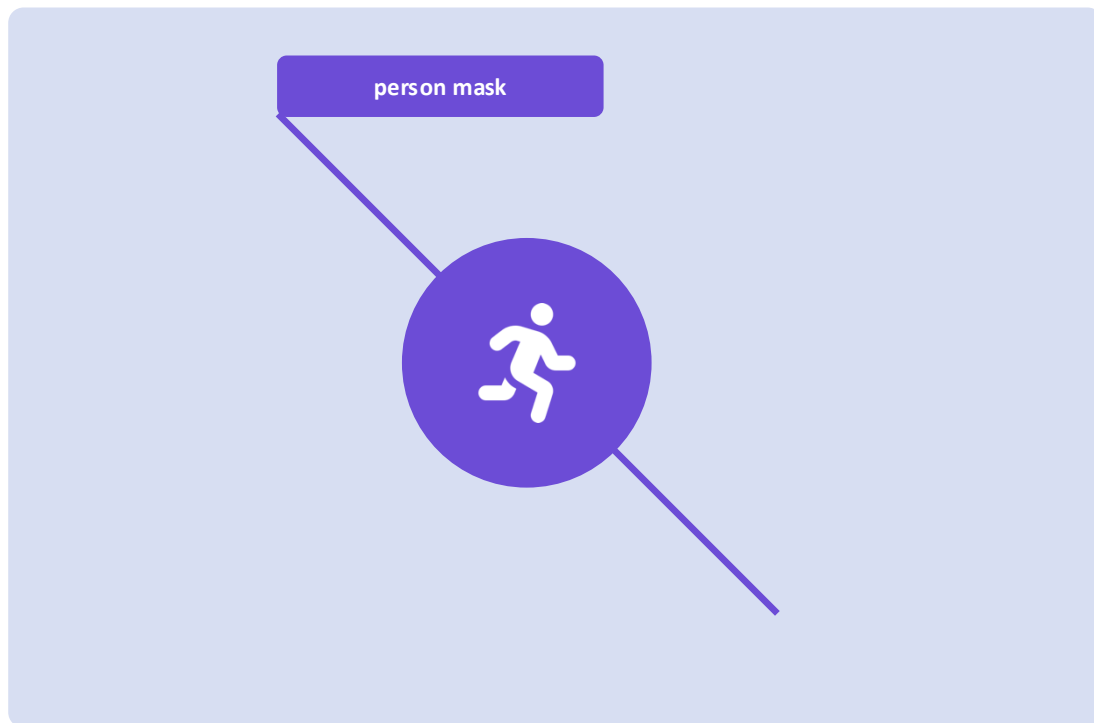
Oriented object detection



Ask AI 



Input & Label: Pixel-Precise Masks



The mask hugs the exact silhouette — not just a rectangle around it.

Semantic vs. Instance segmentation

Instance — one mask per object (this is what Ultralytics gives you).

Semantic — merge all instances of a class into one map (e.g. "all sky pixels").

Label format — variable-length polygon per object

```
<class_id> <x1> <y1> <x2> <y2> ... <xn> <yn>
```

Toy dataset used in class: coco8-seg (same 8 COCO images, with polygon masks)

✦ Roboflow, CVAT, and Segment Anything (SAM)-assisted tools speed up polygon labeling a lot — Ultralytics even ships *ultralytics.SAM* as an auto-labeler.



Where This Task Gets Used



Self-driving cars

Segment road, lane markings, sidewalk, and obstacles pixel-by-pixel.



Portrait / background effects

"Blur the background" or virtual-background effects in video calls.



Land parcel mapping

Outline exact field or building boundaries from satellite tiles.



Medical imaging

Trace the exact boundary of a tumor, organ, or blood vessel for measurement.



Precision agriculture

Separate crop from weed and soil, pixel by pixel, to guide a spraying robot.



Fashion & retail

Cut out a garment precisely for virtual try-on or catalog editing.

Rule of thumb: reach for segmentation when the exact shape or boundary matters — measuring area, precise cutouts, or telling touching objects apart.

Slower to label than boxes — polygon annotation is the trade-off for pixel-level precision.



How We Score It: Two Metrics in One

Segmentation is graded twice — once for the box around each object, once for the mask itself.

Box metrics

Same as detection: mAP50 / mAP50-95, using box IoU. Tells you "did we find the object at all?"

Tells you "how precisely did we trace the shape?" — a model can find the right box but still draw a sloppy outline.

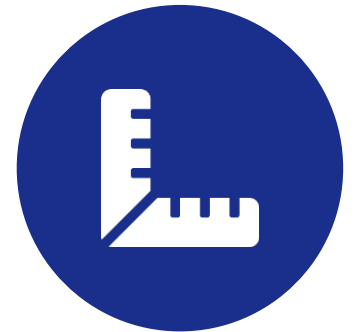
Mask metrics

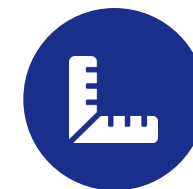
Also mAP50 / mAP50-95, but IoU is computed pixel-by-pixel between the predicted and ground-truth mask instead of box corners.

TASK 5

Depth Estimation

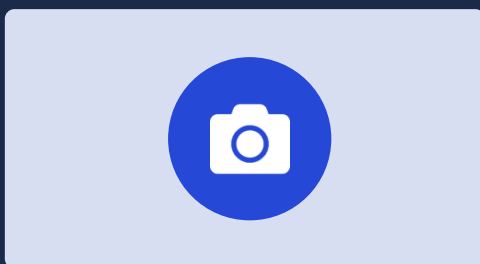
"How far away is each pixel?" — distance from a single 2D photo.



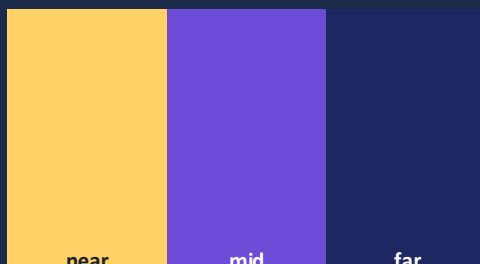


One Photo, No Stereo, No LiDAR

Same scene, two views



RGB input



depth map

A dense encoder-decoder network (e.g. MiDaS, DPT) outputs one distance value for every pixel.

⚠️ YOLO does not cover depth — there is no yolov8n-depth.pt. We use a pretrained MiDaS model instead (zero-shot, no training needed).

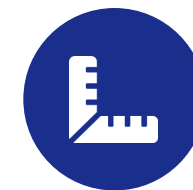
Fine-tuning data, if you want it

```
my_depth_dataset/  
  rgb/      img001.png  
  depth/    img001.png  (16-bit, pixel = distance)
```

RGB + a matching depth map per image — from a sensor (NYU Depth V2, KITTI) or synthetic rendering.

Not a one-line `model.train()` — fine-tuning MiDaS/DPT means writing a small PyTorch loop with a scale-invariant loss (SiLog or masked L1), since real depth sensors leave gaps.

For a class project, zero-shot MiDaS inference is usually enough.



Applications & How We Score It

Where it is used



Portrait mode

Blur the background behind a subject using estimated distance.



Robot & vehicle navigation

Judge how far obstacles are without expensive LiDAR.



Augmented reality

Realistically occlude virtual objects behind real ones.

Metrics (lower = better, except $\delta 1$)

AbsRel

$\text{mean}(|\text{pred} - \text{gt}| / \text{gt})$ — average relative error

RMSE

root-mean-squared error, in depth units

$\delta 1$

fraction of pixels within 25% of the truth — a tolerant "close enough" score (higher = better)

MiDaS predicts relative, unscaled depth (no metric units) — a real evaluation fits a scale + shift to the ground truth before comparing, a detail every monocular-depth paper handles this way.

TASK 6

Pose Estimation

"Where are the joints?" — a skeleton of keypoints per person.



The engine behind the platform. The standard in vision AI.

From YOLOv5 to YOLO26, Ultralytics builds the most widely adopted vision AI models, trusted by millions of developers and enterprises.

Object detection

Instance segmentation

Semantic segmentation

Depth estimation

Image classification

Pose estimation

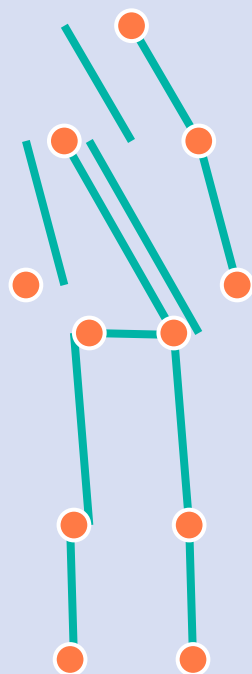
Detect and label body keypoints for body position monitoring.

Oriented object detection





Input & Label: A Skeleton of Keypoints



17 COCO keypoints: nose, eyes, ears, shoulders, elbows, wrists, hips, knees, ankles.

Label format — box + (x, y, visibility) per keypoint

```
<class> <xc> <yc> <w> <h> <x1> <y1> <v1> ... <xk> <yk> <vk>
```

visibility: 0 = not labeled · 1 = labeled but occluded · 2 = labeled and visible

YAML also needs:

```
kpt_shape: [17, 3]  
flip_idx: [0, 2, 1, 4, 3, ...] ← which keypoints swap on a mirror flip
```

Toy dataset used in class: coco8-pose (8 images, standard 17-keypoint skeleton)



Applications & How We Score It



Fitness apps

Count reps and correct exercise form from a phone camera.



Physical therapy

Measure range of motion during rehab exercises remotely.



Sports analytics

Track an athlete's joint angles and movement efficiency.



Fall detection

Recognize a fallen posture for elderly-care monitoring.

How we score it

Box metrics (reused from detection) tell us "did we find the person at all?"

Pose mAP (OKS) replaces box IoU with *Object Keypoint Similarity* — it tolerates more error on naturally wobbly points (like the tip of the nose) than on stable ones (like a hip), because that matches how humans naturally judge a "close enough" pose match.



Choosing the Right Task

Ask yourself what you actually need to know about the image — the task usually falls out naturally.





Things Every CV Student Should Know



2012: the ImageNet moment

AlexNet's huge leap on the ImageNet challenge kicked off the deep-learning era of computer vision — almost every model since builds on that CNN idea.



Data bias is a real risk

A model is only as fair as its training data — under-represented skin tones, angles, or lighting lead to systematically worse predictions for those groups.



Privacy & surveillance concerns

The same detection/pose models that count cars or track fitness reps can track individual people — always consider consent and data protection.



Real-time changed everything

YOLO ("You Only Look Once") made detection fast enough for live video, unlocking self-driving cars, live sports graphics, and phone apps.



Labeling is often the bottleneck

Collecting images is easy; drawing accurate boxes, masks, or keypoints is slow and expensive — this is why tools like Roboflow, CVAT, and SAM matter.



Foundation models are changing the field

Models like SAM (segment anything) and CLIP (image-text understanding) can handle new tasks with little or no fine-tuning at all.



The Full Picture, Side by Side

Task	Model	Toy Dataset	Label Format	Headline Metric
Classification	yolov8n-cls.pt	mnist160	class_folder/image.jpg	Top-1 / Top-5 accuracy
Detection (BBox)	yolov8n.pt	coco8.yaml	class xc yc w h	mAP50 / mAP50-95
OBB	yolov8n-obb.pt	dota8.yaml	class x1 y1 ... x4 y4	mAP50 / mAP50-95 (rotated IoU)
Segmentation	yolov8n-seg.pt	coco8-seg.yaml	class x1 y1 ... xn yn	mask mAP50 / mAP50-95
Depth Estimation	pretrained MiDaS	— (zero-shot)	RGB ↔ depth-map pairs	AbsRel, RMSE, $\delta 1$
Pose	yolov8n-pose.pt	coco8-pose.yaml	box + (x,y,v) per keypoint	pose mAP50-95 (OKS)

The one pattern that repeats everywhere:

```
YOLO(model) → .train(data) → .predict(image) → .val(data)
```



Exercises for Students

1

Swap the dataset

Replace mnist160 with imagenette160 (10 real object classes instead of digits) and retrain classification.

2

Label your own data

Take 20–30 phone photos of 2–3 object classes you care about, label them with Roboflow (free tier), export in YOLO format, and retrain detection on your own data.

3

Study the training curve

Increase epochs on any one task and plot how mAP50-95 changes — where does it start to plateau?

4

Try real depth metrics

Find 5 (image, depth) pairs from NYU Depth V2 and compute real AbsRel / RMSE / $\delta 1$ with the `depth_metrics()` function instead of the synthetic demo.



Questions?

Week 11 · Computer Vision · CP020003 Artificial Intelligence 2026